

ভূমিকা (Introduction)

তৃতীয় অধ্যায়ে আমরা দেখেছিলাম কীভাবে একটি নির্দিষ্ট কাজের জন্য একটি মডেলকে fine-tune করা যায়। তখন আমরা সেই একই tokenizer ব্যবহার করি যেটি দিয়ে মডেলটি আগে থেকে pretrained ছিল। কিন্তু যদি আমরা শুরু থেকে (scratch থেকে) একটি মডেল প্রশিক্ষণ দিতে চাই, তখন কী করব?

এই ধরনের ক্ষেত্রে সাধারণত অন্য কোনো domain বা ভাষার corpus দিয়ে pretrained করা tokenizer ব্যবহার করা ভালো ফল দেয় না। উদাহরণ হিসেবে বলা যায়, যদি কোনো tokenizer ইংরেজি corpus দিয়ে প্রশিক্ষিত হয়, তাহলে সেটি জাপানি টেক্সটের corpus-এ ভালোভাবে কাজ করবে না। কারণ দুই ভাষায় space এবং punctuation ব্যবহারের ধরন একেবারেই আলাদা।

এই অধ্যায়ে আপনি শিখবেন কীভাবে একটি নতুন tokenizer একটি text corpus এর উপর প্রশিক্ষণ দিতে হয়, যাতে সেটি পরে একটি language model pretrain করার জন্য ব্যবহার করা যায়। এই পুরো প্রক্রিয়াটি করা হবে Tokenizers লাইব্রেরির সাহায্যে, যা Transformers লাইব্রেরির “fast” tokenizer গুলো সরবরাহ করে। আমরা এই লাইব্রেরির বিভিন্ন বৈশিষ্ট্য বিস্তারিতভাবে দেখব এবং fast tokenizer ও “slow” tokenizer এর মধ্যে পার্থক্যও বিশ্লেষণ করব।

এই অধ্যায়ে যেসব বিষয় আলোচনা করা হবে:

- একটি নির্দিষ্ট checkpoint-এ ব্যবহৃত tokenizer-এর মতো নতুন tokenizer কীভাবে নতুন text corpus-এ প্রশিক্ষণ দেওয়া যায়
- fast tokenizer-এর বিশেষ বৈশিষ্ট্যগুলো
- বর্তমানে NLP-তে ব্যবহৃত তিনটি প্রধান subword tokenization algorithm-এর পার্থক্য
- Tokenizers লাইব্রেরি ব্যবহার করে শুরু থেকে একটি tokenizer তৈরি করা এবং কিছু ডেটার উপর সেটিকে প্রশিক্ষণ দেওয়া

এই অধ্যায়ে যেখানে কৌশলগুলো আপনাকে অধ্যায় ৭-এর সেই অংশের জন্য প্রস্তুত করবে যেখানে আমরা Python source code এর জন্য একটি language model তৈরি করার বিষয়টি দেখব।